Streaming Packet-to-Flow Feature Pipeline — Kaggle Implementation Guide

A detailed execution plan for building the Stage 1 / Stage 2 pipeline on Kaggle Notebooks, against an 11-day real-world packet capture, under a 30GB RAM ceiling. No code — this is the blueprint to build from.

---

1. Kaggle Environment Constraints (Read This First)

Everything downstream in this guide is shaped by these platform limits, so they're worth stating up front instead of discovering mid-build:

Constraint	Value	Impact on this pipeline
Session execution time	12 hours max per notebook session (CPU or GPU)	An 11-day capture almost certainly **cannot** be processed in one session — the pipeline must be checkpoint-resumable across multiple sessions
Idle timeout (interactive editing)	20 minutes of no activity ends the session	Irrelevant if you run via **Save & Run All** (batch mode) rather than watching an interactive cell — batch mode should be the default execution style for this job
Auto-saved disk (/kaggle/working)	20GB persisted as notebook output	Final/checkpoint files that need to survive between sessions **must** stay small and live here — this is not where you dump raw intermediate data
Scratch disk (outside /kaggle/working)	Extra space available during a session, **not saved** after it ends	Fine for transient per-file processing, but anything here vanishes on session end — never store checkpoint state or partial results only there
Input data (/kaggle/input)	Read-only, mounted from attached Kaggle Datasets	Your 11-day parquet capture needs to be uploaded as a Kaggle Dataset first; you cannot write into this path
RAM	~30GB (verify against the specific instance/accelerator	Confirmed working budget for active_flows sizing (Section 5)

	option you select — this varies by configuration)	
Internet access	Off by default, toggle in Session Options	Only matters if a required library isn't preinstalled; not needed if working entirely with parquet/pandas/pyarrow, which ship by default

The single biggest design consequence of Kaggle specifically: because no session can outlive 12 hours and no large intermediate file can outlive a session unless it's under the 20GB working-directory cap, the pipeline must be restructured around **daily chunks with cross-session checkpointing**, not a single continuous 11-day run. Section 3 below is built around this.

---

2. Directory Layout on Kaggle

```
/kaggle/input//        □ read-only, 11 days of packet parquet files
/kaggle/working/
  checkpoint/
    progress.json              □ which day/file/batch has been completed
  stage1_output/
    flows_day01.csv
    flows_day02.csv
    ...
  stage2_output/
    final_day01.csv
    final_day02.csv
    ...
  logs/
    run_log.txt
```

Splitting output **per day** (rather than one giant growing CSV) is deliberate: each file stays small enough to commit safely within the 20GB cap, and a crash only risks re-doing the current day, not the whole run.

---

3. Multi-Session Execution Strategy (Kaggle-Specific)

Since one run can't cover 11 days, treat each Kaggle notebook session as processing **one or a few calendar days**, then persisting state for the next session to pick up.

3.1 Session lifecycle

1. **Start of session**: load checkpoint/progress.json from /kaggle/working (this survives because it's in the auto-saved directory). If absent, this is day 1, batch 0.
2. **Process**: work through as many days as fit in the time budget (target ~10 hours of actual processing, leaving margin under the 12-hour cap for save/commit overhead).
3. **Before the session ends**: write/update progress.json, ensure all completed days' CSVs are flushed and closed (not left in an open-handle/partial state).
4. **Commit the notebook version** ("Save & Run All" / Save Version) so /kaggle/working — including checkpoint and completed daily outputs — persists as the notebook's output.
5. **Next session**: re-open the notebook (its own prior output is available to itself under /kaggle/working if versioned correctly, or re-attach the previous output as an input dataset) and resume from progress.json.

3.2 Why per-day granularity specifically

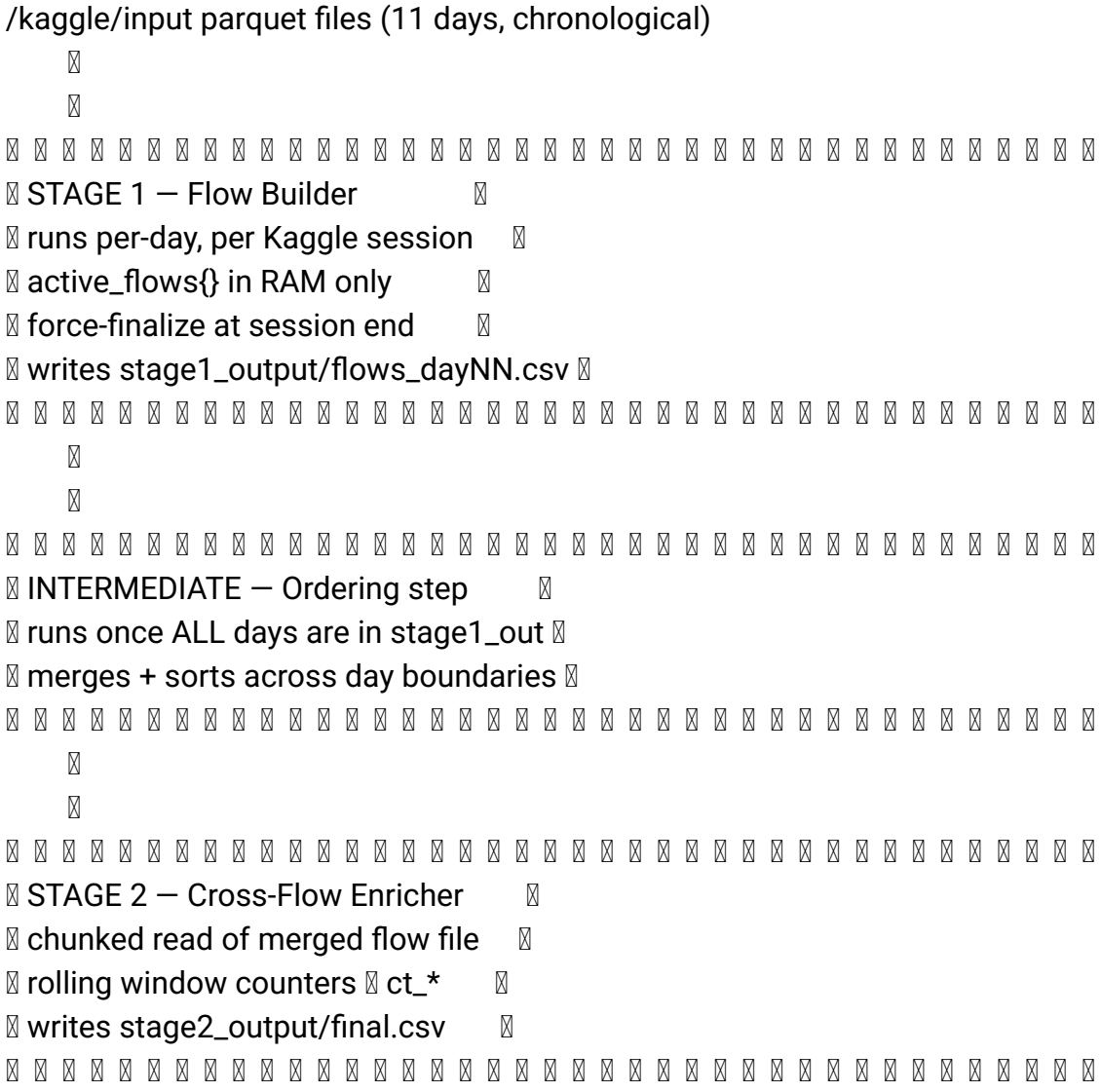
- A single day's worth of flows is small enough to comfortably fit under the 20GB persisted-output cap even after 11 days accumulate (flow-level data is dramatically smaller than packet-level).
- It gives a natural, meaningful resume boundary — "redo at most one day's processing" on crash, not "redo an arbitrary batch offset."
- It lets you validate output quality day-by-day before committing to running the rest — catch a bug on day 1's output instead of discovering it after burning 8 sessions.

3.3 What NOT to rely on

- Don't rely on scratch/non-persisted disk for anything you need in the next session — active_flows dict, in-progress FlowState objects, and any partially-written files outside /kaggle/working are gone when the session ends, checkpoint or not.
- Don't assume a flow that's still "open" (not yet timed out) at the end of a session can be resumed mid-state — the simplest and most robust rule is: **force-finalize all open flows at the end of every session**, and accept that a flow spanning a session boundary gets split into two (this is a small accuracy tradeoff worth taking deliberately rather than trying to serialize/restore live FlowState objects, which is fragile).

---

4. Architecture Overview (Two Stages, Kaggle-Adapted)



Stage 1 runs incrementally across multiple sessions (one day or a few days per session). Stage 2 only starts once **all** days have completed Stage 1 — cross-flow window counts need the full time-ordered flow sequence, so it shouldn't start on partial data.

---

5. Stage 1 — Packet → Flow (Per-Day, Per-Session)

5.1 Input schema (column-projected at read time)

ts, ip_src, ip_dst, t_sport, t_dport, ip_proto, ip_totallen, ip_ttl, t_seqnum, t_acknum, t_flags, t_window

Project only these columns when reading each parquet file — reduces both memory footprint and Kaggle's I/O time against /kaggle/input.

5.2 Output schema (flow-level CSV, per day)

srcip, dstip, sport, dsport, proto, state, service, dur, sbytes, dbytes, sttl, dttl, sloss, dloss, Sload, Dload, Spkts, Dpkts, smeansz, dmeansz, swin, dwin, stcpb, dtcpb, Sjit, Djit, Stime, Ltime, Sintpkt, Dintpkt, tcprtt, synack, ackdat, is_sm_ips_ports — 34 columns, matching the fully-derivable + approximate feature set established earlier.

5.3 Required components

Component	Responsibility
FileManifestBuilder	Lists parquet files for the**current day**only (from /kaggle/input), sorted chronologically
PacketBatchReader	Column-projected batch reader over one file at a time
FlowKeyResolver	Builds the 5-tuple flow key; resolves canonical direction
FlowState	Per-flow running-aggregate accumulator (no raw packet storage)
WelfordAccumulator	Online mean/variance for Sjit/Djit
TcpStateClassifier	Flag-history □state label
ServiceLookup	Port □service string
LossDetector	Duplicate/out-of-order seqnum □sloss/dloss
FlowTable	Wraps active_flows dict;get_or_create,update,evict_expired,force_finalize_all
MemoryMonitor	RSS tracking; triggers emergency eviction if approaching the 30GB soft ceiling
SessionTimeGuard	Tracks elapsed wall-clock time against a configured budget (e.g., 10-hour soft limit, leaving margin under Kaggle's 12-hour hard cutoff); signals graceful wind-down
CheckpointManager	Reads/writes progress.json in /kaggle/working/checkpoint/
FlowCsvWriter	Buffered append-writer to stage1_output/flows_dayNN.csv

5.4 Batching / flush logic

- Batch size tuned via a dry run on one file (start ~250k–500k rows).
- evict_expired runs every batch, not every file.
- SessionTimeGuard checked between files (not mid-file) — cleaner boundary for checkpointing, avoids splitting a file's processing across a checkpoint.
- On SessionTimeGuard trip, or on reaching the end of the current day's files: call force_finalize_all, close the CSV writer, write progress.json, end the session cleanly rather than letting Kaggle's hard 12-hour cutoff kill it mid-write.

5.5 Timeout strategy

- TCP idle timeout (~120s), UDP idle timeout (~60s), and a hard max-duration cap — same three-tier strategy as the general design, calibrated from a single-file dry run.
- Additional Kaggle-specific timeout: **session-boundary force-finalize** (Section 3.3) — every flow still open when a session ends is finalized immediately, regardless of its idle timer. This is a deliberate accuracy tradeoff (a flow spanning midnight-of-session-end gets split into two records) in exchange for never having to serialize live in-memory state across sessions.

5.6 Ordering guarantees

- Within a day: files validated as internally time-sorted and processed in chronological order.
- Across days: since each day is a separate output file, global ordering isn't enforced yet at this stage — that's explicitly deferred to the Intermediate step (Section 6), not assumed here.

5.7 Memory constraints

- Same budget logic as the general design: OS/library overhead (~3-5GB) + active_flows working set (~18-22GB) + buffers/margin (~3-5GB), inside the ~30GB available.
- Processing one day at a time (rather than all 11 days' worth of files in one long-running active_flows table) is itself a memory safety feature — it bounds the realistic maximum number of concurrent flows to "one day's traffic" rather than "eleven days' worth of not-yet-expired edge cases."

5.8 Failure points (Kaggle-specific additions)

Risk	Mitigation
Session hits 12-hour hard	SessionTimeGuard soft limit (e.g., 10hrs) must trigger well

cutoff before graceful wind-down	before the hard cutoff, with buffer for force_finalize_all+ write + checkpoint to complete
Output exceeds 20GB auto-save cap	Per-day CSVs are small individually; monitor cumulative stage1_output/ size each session and archive/download older days if approaching the cap, keeping only recent + unprocessed days live
Scratch (non-persisted) disk used accidentally for checkpoint or output	Explicit path discipline: checkpoint and all outputs must live under /kaggle/working
Notebook disconnects mid-write (not just timeout — could be platform hiccup)	Buffered writer flushes on a row-count threshold, not just at the very end, so a mid-session disconnect loses at most one partial buffer, not the whole day
Burst of unique flows spikes active_flows	MemoryMonitor emergency-flush of oldest entries
Malformed rows	Per-batch validation, quarantine to a reject log rather than crashing the run

5.9 Validation checks (per day)

- Packet-count reconciliation: sum(Spkts + Dpkts) for the day's output ≈ total ingested packets for that day minus quarantined rows.
- Flow-count sanity check (order-of-magnitude expectation) before moving to the next day.
- Spot-check a handful of finalized flows against a slow/naive reference calculation.

---

6. Intermediate Step — Cross-Day Ordering

Runs once, after **all** days have completed Stage 1 (i.e., stage1_output/ contains flows_day01.csv through flows_day11.csv).

- **Option A (strict)**: chunked external merge-sort across all 11 daily files by Ltime, producing one time-ordered file. Given flow-level data is much smaller than packet-level, this is feasible within a single session even at 11 days' combined size — but confirm the merged file size against the 20GB cap; if it's tight, this step can output directly into Stage 2's input stream rather than materializing a full merged file on disk.
- **Option B (relaxed)**: concatenate daily files in day order without a full timestamp sort, accepting that ct_* windows reflect "last 100 flows in day-then-write order" rather than strict global chronological order. This is a legitimate simplification given each day's data is already

internally ordered and days are processed in sequence — the only imprecision is at day boundaries.

Either option should be a conscious choice, logged in progress.json or the run log, not a silent default.

- verify_monotonic_order should run as a gate before Stage 2 begins, regardless of which option is chosen.

---

7. Stage 2 — Cross-Flow Feature Enrichment

Can typically run as a **single session** (or a small number of them) since it operates on much smaller flow-level data rather than raw packets.

7.1 Input schema

The 34-column Stage 1 output, ordered per Section 6.

7.2 Output schema

34 columns + ct_srv_src, ct_srv_dst, ct_dst_ltm, ct_src_ltm, ct_src_dport_ltm, ct_dst_sport_ltm, ct_dst_src_ltm, ct_state_ttl = **42 columns**, written to stage2_output/final.csv.

7.3 Required components

Component	Responsibility
ChunkedCsvReader	Reads the merged flow file in bounded chunks
WindowCounterBank	Fixed-size (100) rolling window + per-key-type counters
WindowCounterBank.query/.insert	Query-before-insert ordering (avoids self-counting)
CrossFlowFeatureComputer	Orchestrates the 8 ct_* computations per row
FinalCsvWriter	Buffered writer to stage2_output/final.csv

SessionTimeGuard/ CheckpointManager	Same pattern as Stage 1, in case the merged file is large enough to need more than one session — checkpoint by row-offset/chunk-index rather than by day

7.4 Memory constraints

- Bounded by one chunk's worth of rows + the fixed 100-entry window — the cheapest stage memory-wise, since window size never scales with total data volume.

7.5 Failure points

Risk	Mitigation
Merged input isn't actually time-ordered	verify_monotonic_order gate (Section 6) before Stage 2 starts
Off-by-one self-counting in window logic	Explicit query-before-insert discipline, tested on a small known sample
Categorical drift between Stage 1's state/ service values and what Stage 2 expects	Validate value vocab before running the full file

7.6 Validation checks

- Brute-force cross-check on a random sample: recompute ct_* via a direct pandas windowed groupby, compare to streaming output.
- Row count in equals row count out (Stage 2 only adds columns).

---

8. How Stage 1 Feeds Stage 2 (Contract Between Stages)

Same principle as the general design, reinforced by Kaggle's session boundaries: **the only thing that crosses from Stage 1 into Stage 2 is the finalized, validated, on-disk CSV** — never in-memory objects. This matters even more on Kaggle than it would on a persistent server, because Stage 1 itself may span several independent sessions with no shared memory between them at all. Stage 2 doesn't know or care how many sessions Stage 1 took — it only consumes the final merged file once Section 6's ordering gate has passed.

---

9. Cross-Cutting Concerns

- **Config file** (/kaggle/working/config.json or similar): timeout values, batch size, chunk size, window size, SessionTimeGuard soft-limit hours, memory soft-ceiling — tuned once via a single-file/single-day dry run, then reused across all sessions so behavior stays consistent.
- **Logging**: append to logs/run_log.txt (not overwrite) each session, so a multi-session run has one continuous audit trail instead of fragments.
- **Resumability**: progress.json is the single source of truth for "what's done" — every session reads it first, writes it last.
- **Idempotency**: if a session dies before updating progress.json but after partially writing a day's CSV, the next session should detect the incomplete day (e.g., missing an end-of-day marker) and safely re-run that day from scratch rather than appending to a possibly-corrupt partial file.
- **Dry run first**: before committing to an 11-day, multi-session run, do one full end-to-end pass (Stage 1 → Intermediate → Stage 2) on a single day's data to validate schema, tune all config values, and confirm output sizes — this is cheap insurance against discovering a bug on day 9 of an 11-day process.

---

10. Suggested Build Order

1. FlowKeyResolver, FlowState, WelfordAccumulator — the core per-flow math, testable on a tiny sample file with no Kaggle-specific concerns yet.
2. FlowTable + evict_expired + force_finalize_all — timeout/session-boundary logic.
3. SessionTimeGuard + CheckpointManager — Kaggle multi-session scaffolding, tested with an artificially short time budget first (e.g., 2 minutes) to confirm resume behavior works before trusting it at 10 hours.
4. Full Stage 1 on one real day → validation checks (Section 5.9).
5. Intermediate ordering step, once a few days exist.
6. Stage 2 components, tested on the small merged sample first.

Say the word when you're ready to turn any of these into actual notebook cells.